

Chapter 3

파드(Pods)

쿠버네티스에서 컨테이너 실행하기

발표자: 개발자 이프

2026

오늘 다룰 내용

1. 왜 파드인가? (3.1)
2. 파드 YAML 해부 (3.2)
3.  **Demo 1:** 파드 라이프 한 바퀴
4. Labels & Selectors (3.3, 3.4)
5.  **Demo 2:** 라벨/셀렉터 구독
6. nodeSelector (3.5) + Annotations (3.6)
7. Namespaces (3.7)
8.  **Demo 3:** ns 격리의 진실
9. 삭제 (3.8)
10.  **Demo 4:** 삭제 패턴 3+1가지
11. 종합

이전 발표와의 연결

[3장] 파드 = K8s의 가장 작은 배포 단위

↓

[4장] 파드를 자동 관리 → RC/ReplicaSet

↓

[5장] 파드 무리에 접근 → Service

↓

[12장] 누가 API 접근 가능? → RBAC

↓

[13장] 파드의 권한/통신 통제 → SecurityContext/PSA/NetworkPolicy

↓

[16장] 순서/이름 필요한 파드 → StatefulSet

→ 모든 K8s 워크로드는 결국 파드를 만든다.

Part 1

왜 파드인가?

컨테이너 하나면 안 되나?

도커와 K8s의 차이

도커: 컨테이너 = 배포 단위

K8s: 파드 = 배포 단위 (≥ 1 개 컨테이너 묶음)

질문: 왜 굳이 묶었나?

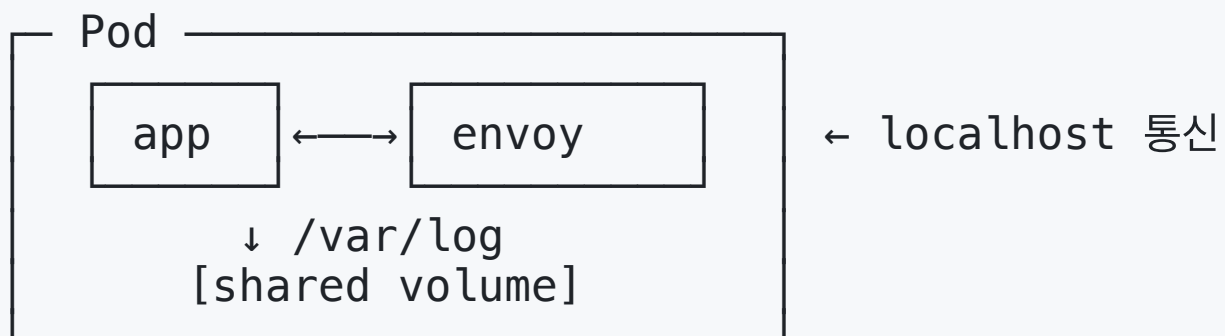
파드가 공유하는 것

같은 파드의 컨테이너들은:

-  같은 IP 주소 (Network namespace 공유)
-  localhost로 통신 가능
-  같은 IPC namespace (System V IPC, POSIX shared memory)
-  같은 hostname (UTS namespace)
-  볼륨 공유 가능 (선언만 하면)
-  Filesystem은 별도 (Mount namespace는 각자)
-  PID namespace는 기본 별도 (옵션으로 공유 가능)

 localhost 통신 + 볼륨 공유가 파드의 핵심 가치.

왜 묶음인가 — 가장 흔한 예: sidecar



- **app**: 비즈니스 로직
- **envoy**: 사이드카 프록시 (mTLS, 라우팅, 로깅)

→ 둘은 같은 파드. 함께 살고 함께 죽음.

묶지 말아야 할 때

언제 한 파드에 묶으면 안 되나:

- ✗ 독립적으로 스케일하고 싶을 때 (web과 worker)
- ✗ 라이프사이클이 다를 때 (배치 잡 + DB)
- ✗ 다른 노드에 분산하고 싶을 때

원칙: 함께 죽어도 OK한 것만 한 파드로.

Part 2

파드 YAML 해부

책 3.2

파드 YAML의 4부분

```
apiVersion: v1          # ① API 버전
kind: Pod               # ② 객체 종류
metadata:               # ③ 식별 정보
  name: hello
  namespace: k8s-demo
  labels:
    app: hello
spec:                   # ④ 원하는 상태 (사용자가 정의)
  containers:
    - name: nginx
      image: nginx:1.27
# status:               # (서버가 채움, 사용자가 쓰지 않음)
```

apiVersion의 의미

apiVersion	들어있는 객체
v1	core API: Pod, Service, Namespace, ConfigMap, Secret, PV/PVC, Node
apps/v1	Deployment, StatefulSet, DaemonSet, ReplicaSet
batch/v1	Job, CronJob
networking.k8s.io/v1	NetworkPolicy, Ingress
rbac.authorization.k8s.io/v1	Role, ClusterRole, RoleBinding
policy/v1	PodDisruptionBudget

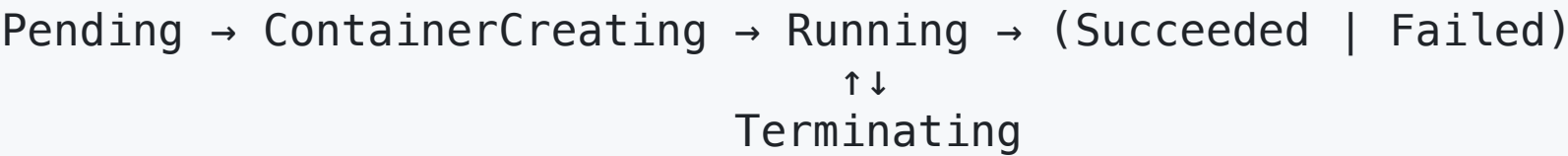
→ Pod는 **core(v1)**. Deployment는 **apps/v1**. 혼동 주의.

kubectl apply vs create

	<code>kubectl create -f X.yaml</code>	<code>kubectl apply -f X.yaml</code>
신규 생성	✓	✓
이미 있으면?	✗ AlreadyExists	✓ patch로 업데이트
변경 추적	✗	✓ last-applied annotation
3-way merge	✗	✓

💡 선언적 운영하려면 항상 `apply`. `create` 는 한 번만 만들 때.

파드 라이프 사이클



`kubectl get pods` 의 STATUS 컬럼이 이 상태:

STATUS	의미
Pending	스케줄 대기 또는 이미지 풀 중
ContainerCreating	컨테이너 시작 중
Running	정상 동작
Error / CrashLoopBackOff	실패 + 재시작 백오프
Terminating	삭제 진행 중 (grace period)
Succeeded / Completed	정상 종료 (Job 등)

READY와 STATUS의 차이

NAME	READY	STATUS	RESTARTS	
hello	1/1	Running	0	
hello	0/1	Running	0	← 컨테이너는 떴지만 readinessProbe 실패
hello	1/1	Running	3	← 3번 재시작 — 왜?

- **READY:** 트래픽 받을 준비됐는지 (readinessProbe)
- **STATUS:** 컨테이너 프로세스 상태 (liveness/실행)
- **RESTARTS:** 컨테이너 재시작 횟수 (liveness 실패 시 증가)

→ 항상 세 컬럼 다 봐야 함.

Demo 1

파드 라이프 한 바퀴

kubectl 6개 명령으로 파드 일생을

Demo 1: 명령 순서

```
# 1. 생성
kubectl apply -f demos/ch03-pods/01-pod-basic.yaml

# 2. 관찰
kubectl get pods -n k8s-demo
kubectl get pods -n k8s-demo -o wide    # IP, 노드 함께
kubectl describe pod hello -n k8s-demo

# 3. 로그
kubectl logs hello -n k8s-demo
kubectl logs hello -n k8s-demo --previous    # 이전 컨테이너 (재시작 후)
kubectl logs hello -n k8s-demo -f           # follow

# 4. 접근
kubectl port-forward pod/hello -n k8s-demo 8080:80 &
curl localhost:8080 | head -3

# 5. 진입
kubectl exec -it hello -n k8s-demo -- sh
# > env | grep KUBERNETES
# > exit

# 6. 삭제
kubectl delete -f demos/ch03-pods/01-pod-basic.yaml
```


Demo 1 — 디테일 짚기

`kubectl describe pod hello` 의 출력 구조:

```

Name:          hello
Namespace:     k8s-demo
Status:        Running
IP:            10.40.x.x
Containers:
  nginx:
    State:      Running
    Ready:      True
    Restart Count: 0
    Image:      nginx:1.27-alpine
Events:
  Type    Reason      Age    From    Message
  ----    -
  Normal  Scheduled   10s    kubelet Successfully assigned ...
  Normal  Pulling     9s     kubelet Pulling image ...
  Normal  Pulled      7s     kubelet Successfully pulled ...
  Normal  Created     7s     kubelet Created container nginx
  Normal  Started     7s     kubelet Started container nginx
  
```

← 💡 디버깅 핵심

Part 3

Labels & Selectors

책 3.3, 3.4

Labels — 그냥 key/value 쌍?

```
metadata:  
  labels:  
    app: web  
    env: prod  
    tier: frontend  
    version: v1.2.3
```

규칙:

- 키: prefix(선택, 253자) + / + name(63자)
- 값: 63자 이하, 영숫자/_/-/. 시작.끝은 영숫자
- 객체당 여러 라벨 가능 (제한 없음)

💡 라벨은 그냥 메타데이터가 아니라 **selector**의 입력.

표준 권장 라벨 (app.kubernetes.io/...)

```
labels:  
  app.kubernetes.io/name: redis  
  app.kubernetes.io/instance: redis-cart  
  app.kubernetes.io/version: "5.0.4"  
  app.kubernetes.io/component: cache  
  app.kubernetes.io/part-of: ecommerce  
  app.kubernetes.io/managed-by: helm
```

→ Helm, ArgoCD, Lens 같은 도구가 이 라벨로 객체 그룹핑.

💡 새 워크로드 만들 때 이 라벨 패턴 권장 — 운영 도구 호환 .

Selector — Labels로 객체 찾기

Equality-based:

```
kubectl get pods -l env=prod          # env이 prod인 것
kubectl get pods -l env!=prod         # env이 prod 아닌 것
kubectl get pods -l env=prod,app=web  # AND
```

Set-based:

```
kubectl get pods -l 'env in (prod,staging)'
kubectl get pods -l 'env notin (dev)'
kubectl get pods -l 'env'                # 라벨 존재
kubectl get pods -l '!unused'            # 라벨 없음
```

→ CLI에서는 같은 -l 옵션, 표현식만 다름.

Selector가 쓰이는 곳

객체	selector 용도
Service	어느 파드로 트래픽 보낼지
Deployment	관리할 파드
ReplicaSet	복제 대상 파드
DaemonSet , StatefulSet , Job	동일
NetworkPolicy	적용 대상 + 통신 허용 대상
PodDisruptionBudget	보호 대상
kubectl -l	CLI 필터

→ K8s에서 객체 매칭은 거의 다 selector.

selector matchLabels vs matchExpressions

YAML 안에서는 두 가지:

```
# Service / DaemonSet 등 - 단순 equality
selector:
  app: web
  env: prod

# Deployment / NetworkPolicy 등 - 확장 가능
selector:
  matchLabels:
    app: web
  matchExpressions:
    - { key: env, operator: In, values: [prod, staging] }
    - { key: unused, operator: DoesNotExist }
```

→ `matchExpressions` 가 set-based selector의 YAML 형태.



Demo 2

Labels & Selectors 구독

Demo 2: 명령 순서

1. 4개 파드 한 번에

```
kubectl apply -f demos/ch03-pods/02-labeled-pods.yaml
```

```
kubectl get pods -n k8s-demo --show-labels
```

2. equality

```
kubectl get pods -n k8s-demo -l env=prod # web-prod, api-prod
```

```
kubectl get pods -n k8s-demo -l env=prod,app=web # web-prod 만
```

3. set-based

```
kubectl get pods -n k8s-demo -l 'env in (prod,staging)' # prod 파드 2개
```

```
kubectl get pods -n k8s-demo -l '!unused' # 모든 파드
```

4. 라벨 수정

```
kubectl label pod web-prod -n k8s-demo tier=frontend
```

```
kubectl label pod web-prod -n k8s-demo env=staging --overwrite
```

```
kubectl get pods -n k8s-demo --show-labels # 변경 확인
```

5. selector로 삭제

```
kubectl delete pod -n k8s-demo -l env=staging # web-prod만 삭제
```

```
kubectl get pods -n k8s-demo
```

6. 전체 정리

```
kubectl delete -f demos/ch03-pods/02-labeled-pods.yaml --ignore-not-found
```

Demo 2 — 흔히 놓치는 디테일

- `--overwrite` 없이 `kubectl label` 을 기존 키에 쓰면 에러

```
error: 'env' already has a value (prod), and --overwrite is false
```

- 라벨 삭제: `kubectl label pod X env-` (키 끝에 `-`)

```
kubectl label pod web-prod -n k8s-demo tier-
```

- `--show-labels` 로 모든 라벨 한눈에
- `-L env,app` 로 특정 라벨을 컬럼으로 표시:

NAME	READY	STATUS	ENV	APP
web-prod	1/1	Running	prod	web

Part 4

nodeSelector (3.5)

노드 라벨 + nodeSelector

노드도 라벨 가짐:

```
kubectl get nodes --show-labels  
kubectl label node <node-name> disk=ssd
```

파드에 `nodeSelector` 로 노드 지정:

```
spec:  
  nodeSelector:  
    disk: ssd  
  containers:  
  - name: app  
    image: ...
```

→ 매칭되는 노드에만 스케줄.

nodeSelector의 한계

- ✗ 표현력 약함 (equality만)
- ✗ "선호하지만 필수 아님" 표현 불가
- ✗ topology 기반 분산 불가

→ **nodeAffinity + podAntiAffinity**가 후속작:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - { key: disk, operator: In, values: [ssd, nvme] }
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
            - { key: zone, operator: In, values: [a, b] }
```



Part 5

Annotations (3.6)

Labels vs Annotations

	Labels	Annotations
용도	객체 식별, selector	메타데이터 보관
형식	짧은 key/value	더 긴 텍스트 OK (JSON 등도)
selector로 검색	✔	✘
길이 제한	value 63자	사실상 무제한
변경	자주 변경 가능	변경 가능하지만 보통 정적

💡 "검색에 쓰면 라벨, 그냥 보관이면 annotation".

Annotations — 흔한 용도

```
metadata:
  annotations:
    # 빌드 정보
    build.git-commit: "a1b2c3d"
    build.timestamp: "2026-06-04T10:00:00Z"

    # 도구별 메타데이터
    kubectl.kubernetes.io/last-applied-configuration: '...'
    deployment.kubernetes.io/revision: "3"
    helm.sh/chart: "myapp-1.0.0"

    # 운영 정보
    contact: "team-platform@example.com"
    description: "User-facing web app"

    # 컨트롤러 동작 제어
    nginx.ingress.kubernetes.io/rewrite-target: "/"
    prometheus.io/scrape: "true"
```


Part 6

Namespaces (3.7)

Namespace = K8s의 폴더

K8s 객체를 그룹핑하는 단위:

- 같은 ns 안에서는 이름 유일 (Pod `web` 하나만)
- 다른 ns에는 같은 이름 가능 (각 ns에 `web` 따로)
- ns 자체는 클러스터 전역 객체 (ns 안에 또 다른 ns ❌)

```
kubectl get namespaces
# default      Active    90d
# kube-system  Active    90d
# kube-public   Active    90d
# k8s-demo     Active    30d
```

시스템 ns 4종

ns	용도
default	명시 안 했을 때의 기본값
kube-system	K8s 자체 컴포넌트 (CoreDNS, kube-proxy, ...)
kube-public	모든 사용자가 읽기 가능 (대부분 비어있음)
kube-node-lease	노드 heartbeat 객체

 kube-system은 절대 만지지 말 것. 클러스터 깨질 수 있음.

Namespace는 격리가 아니다

진실:

-  이름 충돌 방지
-  ResourceQuota / LimitRange 단위
-  RBAC 권한 단위
-  네트워크 격리 X (기본은 다 통함)
-  이미지 풀 격리 X
-  노드/디스크 격리 X

격리 원하면:

- 네트워크: **NetworkPolicy** (13장)
- 권한: **RBAC** (12장)
- 자원: **ResourceQuota** (이번 발표 범위 밖)

Cross-Namespace 통신

다른 ns의 서비스 접근:

```
<service-name>.<namespace>.svc.cluster.local
```

같은 ns:

```
curl http://api                # 짧게 가능  
curl http://api.k8s-demo.svc.cluster.local  # 풀 이름
```

다른 ns:

```
curl http://api.other-ns.svc.cluster.local
```

→ DNS만으로 접근 가능 (NetworkPolicy 없으면).

kubectl context와 namespace

매번 `-n` 치기 귀찮으면:

```
# 현재 context의 기본 ns 변경
kubectl config set-context --current --namespace=k8s-demo

# 확인
kubectl config view --minify | grep namespace

# 임시 변경 (한 명령에만)
kubectl get pods -n other-ns
```

💡 **kubens** 같은 도구로 간편 전환 가능. 또는 alias 만들기:

```
alias kns='kubectl config set-context --current --namespace'
kns k8s-demo
```

함정: default ns의 위험

신규 K8s 사용자가 자주 빠지는:

1. **default ns에 prod 워크로드** → RBAC/quota 분리 불가
2. 여러 환경(**dev/staging/prod**)을 같은 ns에 → 라벨로 분리해도 RBAC 분리 안 됨
3. 모든 사용자에게 **default ns 권한** → 한 사람이 다른 사람 워크로드 건드림

해결:

- ns를 환경(**dev/staging/prod**) 또는 팀(**team-a/team-b**) 단위로 분리
- 각 ns에 RBAC + ResourceQuota + NetworkPolicy

Demo 3

Namespace 격리의 진실

Demo 3: 명령 순서

```
# 1. 두 ns + 각 ns에 echo 파드/서비스 (한 매니페스트)
kubectl apply -f demos/ch03-pods/03-cross-ns-pods.yaml

# 2. 전체 보기
kubectl get pods,svc -A | grep ch03-demo

# 3. 기본 ns 변경
kubectl config set-context --current --namespace=ch03-demo-a
kubectl get pods    # prefix 없이 ch03-demo-a 거만

# 4. 핵심: cross-ns 통신
kubectl exec -n ch03-demo-a echo -- wget -q0- echo.ch03-demo-b.svc.cluster.local | head -3
# → <html>...nginx welcome page
# ⚠ 다른 ns의 서비스에 그냥 접근됨. 격리 아님.

# 5. 컨텍스트 복귀
kubectl config set-context --current --namespace=default

# 6. (Demo 4에서 ns 삭제로 정리할 예정 - 일단 그대로 둬)
```

Part 7

파드 삭제 (3.8)

삭제의 4가지 패턴

패턴	명령	용도
By name	<code>kubectl delete pod hello</code>	단일 파드
By selector	<code>kubectl delete pod -l env=dev</code>	라벨 그룹
All in ns	<code>kubectl delete pod --all -n k8s-demo</code>	ns 안 전체
Cascade by ns	<code>kubectl delete ns k8s-demo</code>	ns + 내부 전부

💡 마지막 패턴이 가장 강력하고 가장 위험. 실수로 prod ns 지우는 사고 흔함.

Graceful 종료 흐름

`kubectl delete pod X` 실행하면:

1. API server: pod에 DeletionTimestamp 설정
2. kubelet: 컨테이너에 SIGTERM 전송
3. 컨테이너 정리할 시간 (기본 30초)
 - 진행 중인 요청 처리, 연결 정리
4. grace period 만료 → SIGKILL
5. 컨테이너 강제 종료
6. API server에서 파드 객체 제거

`terminationGracePeriodSeconds` 로 조정:

```
spec:  
  terminationGracePeriodSeconds: 60    # 기본 30
```

`--force --grace-period=0` — 응급 옵션

```
kubectl delete pod hello --grace-period=0 --force
```

동작:

1. API server에서 즉시 파드 객체 제거
2. **kubelet**은 모름 → 컨테이너 정리 안 됨 (orphan)
3. 노드에 컨테이너 남아있을 수 있음

⚠ 정말 **stuck**된 경우에만. 평소엔 ❌.

언제 정당한가:

- 노드가 unreachable이라 정상 삭제 실패
- finalizer 때문에 hang된 상태

Finalizer로 인한 삭제 hang

파드가 Terminating 상태에서 안 사라지면:

```
kubectl get pod X -o jsonpath='{.metadata.finalizers}'  
# ["external-cleanup-controller/cleanup"]
```

→ 외부 컨트롤러가 cleanup 끝나야 삭제 진행. 컨트롤러가 죽었으면 영원히 hang.

응급:

```
kubectl patch pod X -p '{"metadata":{"finalizers":null}}' --type=merge
```

⚠ 강제 finalizer 제거 = cleanup 안 됨 (외부 자원 누수 가능).

Demo 4

삭제 패턴 3+1가지

Demo 4: 명령 순서

```
# 0. 시연용 파드 준비
kubectl apply -f demos/ch03-pods/04-delete-targets.yaml
kubectl get pods -n k8s-demo

# 패턴 1: by name
kubectl delete pod target-1 -n k8s-demo
kubectl get pods -n k8s-demo

# 패턴 2: by selector
kubectl delete pod -n k8s-demo -l role=tobedeleted
kubectl get pods -n k8s-demo

# 패턴 3: --all (k8s-demo의 모든 파드)
kubectl delete pod --all -n k8s-demo

# 패턴 4: namespace cascade (Demo 3 잔재 정리)
kubectl delete ns ch03-demo-a ch03-demo-b --ignore-not-found

# 응급 (시연만 – 안 쓰는 게 정답)
# kubectl delete pod <stuck> --grace-period=0 --force
```


Demo 4 — Graceful 동작 관찰

-w 옵션으로 STATUS 전이 보기:

한 터미널에서:

```
kubectl get pods -n k8s-demo -w
```

다른 터미널에서:

```
kubectl delete pod target-2 -n k8s-demo
```

첫 터미널 출력:

NAME	READY	STATUS	AGE	
target-2	1/1	Terminating	30s	← grace period 동안
target-2	0/1	Terminating	30s	
target-2				← 완전히 사라짐

→ Terminating 이 약 30초간 보임 (기본 grace period).

Part 8

종합

흔히 놓치는 디테일 모음

#	디테일
1	<code>kubectl apply</code> 항상, <code>create</code> 는 일회성만
2	<code>STATUS=Running</code> 이라도 <code>READY=0/1</code> 이면 트래픽 ❌
3	<code>describe</code> 의 <code>Events</code> 가 디버깅 시작점
4	라벨 prefix <code>app.kubernetes.io/...</code> 로 도구 호환성
5	<code>kubectl get -L env,app</code> 로 컬럼 표시
6	<code>kubectl label X key-</code> 로 라벨 삭제
7	<code>nodeSelector</code> 는 equality만 — <code>affinity</code> 가 후속
8	<code>Labels</code> = 검색용 / <code>Annotations</code> = 보관용
9	<code>Namespace</code> 는 격리 아님 — <code>RBAC</code> + <code>NetworkPolicy</code> 필요
10	<code>default ns</code> 에 <code>prod</code> 워크로드 ❌

1판 vs 현실 차이 정리

1판 내용	현실 (2026 기준)
<code>kubectl run --restart=Never</code> 로 파드 생성	가능하지만 deprecated 경고 — YAML 권장
Pod 객체 그대로 사용	보통 Deployment/StatefulSet 통해 (4장+)
<code>nodeSelector</code>	<code>nodeAffinity</code> 가 더 자주 쓰임
<code>kubectl create -f</code>	<code>kubectl apply -f</code> 권장
(없음)	sidecar 컨테이너 native 지원 (1.28 GA)
(없음)	ephemeral containers (debug용, 1.25 GA)

💡 책 보면서 `kubectl run` 부분은 "지금은 YAML+apply".

다음 발표 예고: 4장 (ReplicationController/ReplicaSet)

3장에서 우리는:

- 파드를 직접 만들고 관리

4장의 문제:

- 파드 죽으면? → 자동 재생성 누가?
- 같은 파드를 3개 띄우려면? → 일일이 만들기?

답: **ReplicationController / ReplicaSet.**

- 원하는 복제본 수 유지
- 파드 죽으면 자동 재생성
- 라벨 selector로 관리 대상 추적 ← 3장의 selector 다시 등장

→ 3장의 Labels & Selectors가 4장의 기반.

Q&A

 감사합니다